FLASHWINDOW: Temporal Evidence Reconstruction and Adaptive Witnessing for Ephemeral Software-Update Tampering

A Unified Architecture for Detecting Transient Configuration Exploits in Software Supply Chains

SHOAIB ARSHAD*

Department of Computing, Riphah International University, Faisalabad, Pakistan, 31973@riphahfsd.edu.pk

(*Corresponding author)

SABA FIRDOUS

Department of Computing, Riphah International University, Faisalabad, Pakistan, saba.firdous@riphahfsd.edu.pk

ZUNAIR-UL-ABIDEN

Department of Computing, Riphah International University, Faisalabad, Pakistan, 31576@riphahfsd.edu.pk

TALHA AHSAN

Department of Computing, Riphah International University, Faisalabad, Pakistan, talha.ahsen@riphahfsd.edu.pk

Software supply chain adversaries increasingly deploy ephemeral configuration tampering. Attackers compromise remote update manifests for brief temporal windows and revert modifications before periodic scanners execute. Point-in-time scanners exhibit an acute visibility gap during transient exposure windows. This paper presents FLASHWINDOW, an external monitoring and evidence-preservation framework for vulnerable software update infrastructure. The architecture decouples deterministic invariant enforcement from an adaptive, bounded artificial intelligence assistance plane. A four-layer evidence core combines lightweight multi-signal probing, temporal tracking, deterministic security invariants, and hash-chained ledgers anchored via checkpoint witnesses. Bounded cost-aware bandit scheduling optimizes request budgets while preserving strict deterministic policy boundaries. Empirical validation spans 500 controlled campaigns, 2,000 benign controls, 100 multi-vantage trials, and 1,375 curated proxy invariant fixtures. Controlled campaigns demonstrate 98.4% detection recall [96.9%, 99.2%] and a 0.35% challenge alert rate [0.17%, 0.72%] across protocol challenge cases. Deterministic invariant evaluation achieves 100.0% recall [99.5%, 100.0%] and 0.0% false alarms across 1,375 curated proxy fixtures. Multi-vantage consensus resolves geographically targeted split-view exploits with 99.0% recall at mean 1.50 seconds modeled latency. Component ablation confirms that hybrid signal-triggered probing reduces network egress bandwidth by 93.6% relative to 10-minute periodic polling. Prototype in-memory invariant processing measures 0.45 ms median latency (0.62 +/- 0.79 ms mean), with 2.68 ms at the 99th percentile. These results demonstrate that temporal witnessing and deterministic invariant verification reliably expose transient supply-chain manipulation without imposing prohibitive network overhead.

CCS CONCEPTS • Security and privacy ~ Software and application security • Security and privacy ~ Distributed systems security • Security and privacy ~ Intrusion detection systems • Computing methodologies ~ Artificial intelligence

Additional Keywords and Phrases: Software Supply Chain Security, Ephemeral Tampering, Update Integrity, Temporal Monitoring, Split-View Detection, Tamper-Evident Logs, External Witnessing, Contextual Bandits

ACM Reference Format:
Shoaib Arshad, Saba Firdous, Zunair-Ul-Abiden, and Talha Ahsan. 2026. FLASHWINDOW: Temporal Evidence Reconstruction and Adaptive Witnessing for Ephemeral Software-Update Tampering. ACM Trans. Priv. Sec., Manuscript submitted to ACM.

1 INTRODUCTION

Modern computing environments depend heavily upon automated software update mechanisms. Enterprise operating systems, database management engines, and cloud services continuously ingest remote configuration updates. Remote manifests direct client systems toward new executable binaries, dynamic libraries, and deployment scripts. When update distribution operates smoothly, vital security vulnerabilities receive immediate patching. However, this automated mechanism introduces severe operational vulnerabilities. Granting automated installation authority to remote update endpoints creates high-value targets for advanced persistent threats.

Sophisticated supply-chain attackers exploit software distribution channels through transient manipulation [1, 12, 16, 56]. Recent telemetry surveys across npm and PyPI demonstrate widespread injection of malicious typosquats and dependency confusion scripts [37, 38, 39, 40, 85]. In October 2026, threat researchers uncovered widespread configuration tampering targeting enterprise utility update infrastructure [1]. Adversaries compromised update configuration endpoints for brief temporal intervals ranging from thirty seconds to fifteen minutes. The attackers served trojanized binaries exclusively during these narrow windows. The threat actors then restored legitimate manifests immediately afterward. Standard periodic scanners execute once every several hours. Consequently, point-in-time monitors fail completely to capture these transient attacks.

This defensive blind spot constitutes the acute temporal visibility gap. Existing endpoint integrity solutions assume that compromised states persist across extended audit windows. When an adversary limits attack dwell time below scanning intervals, detection probability collapses toward zero. Passive network sniffers examine traffic payloads but cannot inspect encrypted TLS handshakes without private key access [9]. Emerging provenance initiatives like in-toto [4] and SLSA [8] enforce build pipeline integrity but omit runtime distribution witnessing. Sigstore transparency logs record signed metadata without monitoring ephemeral mirror state [6]. Furthermore, client-side update frameworks such as TUF [2] and Uptane [3] require client modifications. Legacy enterprise updaters cannot easily be recompiled to integrate complex cryptographic attestation. Therefore, external witnessing without client modification remains essential.

This paper proposes FLASHWINDOW, a unified external monitoring and evidence-preservation framework for vulnerable update infrastructure. FLASHWINDOW rejects the assumption that point-in-time scans provide adequate visibility. Instead, the framework deploys continuous multi-signal micro-probes combined with Poisson background sampling. A four-layer evidence core bridges lightweight observation and forensic immutability. The architecture decouples deterministic invariant enforcement from an adaptive, bounded artificial intelligence assistance plane. Deterministic rules govern critical alerting, while bounded algorithms optimize request budgets and triage ambiguous anomalies.

The primary scientific contributions of this investigation are four-fold:

1) Formal Threat Modeling and Visibility Gap Analysis: A comprehensive formalization defines transient update tampering across four trust boundaries. Mathematical models quantify detection probability limits under discrete and Poisson sampling.

2) Four-Layer Evidence Core and Decoupled AI Plane: A layered architecture couples lightweight multi-signal probing with state-machine reconstruction, deterministic invariants, and hash-chained ledgers. Bounded AI assistance optimizes scheduling without alert suppression.

3) Multi-Vantage Consensus and Split-View Resolution: A distributed cross-verification algorithm detects geographically targeted and ASN-selective payload delivery. Consensus arbitration resolves split-view divergence within mean 1.50 seconds.

4) Empirical Validation across Benchmark Corpora: Rigorous empirical evaluation spans 500 controlled transient tampering campaigns alongside 1,375 curated proxy invariant benchmark trials. FLASHWINDOW demonstrates 98.4% detection recall [96.9%, 99.2%], a 0.35% challenge alert rate [0.17%, 0.72%], 100.0% invariant recall, and 0.45 ms median latency.

To systematically validate these guarantees, this work formulates four central research questions:

- **RQ1 (Detection Efficacy):** How reliably does hybrid multi-signal probing capture transient update tampering spanning 30 seconds to 15 minutes compared to periodic baselines?
- **RQ2 (Overhead Reduction):** What reduction in network egress bandwidth does signal-triggered probing achieve relative to continuous full manifest inspection?
- **RQ3 (Multi-Vantage Resolution):** How effectively does distributed cross-vantage verification detect split-view manipulation targeting specific autonomous systems?
- **RQ4 (Forensic Immutability):** Does cryptographic Merkle receipt anchoring provide verifiable evidence that withstands post-incident tampering?

The remainder of this paper is structured as follows. Section 2 reviews existing literature and establishes state-of-the-art comparison boundaries. Section 3 formalizes the adversarial threat model and security invariants. Section 4 presents the FLASHWINDOW four-layer evidence architecture. Section 5 develops the mathematical sampling model and analytical detection proofs. Section 6 describes the implementation pipeline and reference algorithms. Section 7 presents empirical benchmark results and ablation studies. Section 8 evaluates operational threats to validity and ethical evidence handling. Section 9 concludes the paper.

2 BACKGROUND AND RELATED WORK

Securing software update distribution requires addressing multiple technological domains. This section reviews prior work in supply-chain security, update frameworks, temporal monitoring, and AI-assisted security triage.

2.1 Ephemeral Supply-Chain Tampering

Software supply-chain attacks have evolved from static repository poisoning to dynamic distribution hijacking [12, 16, 25]. In package registries like npm and PyPI, attackers inject typosquatted packages and dependency confusion vectors [37, 38, 39, 40, 67]. Modern infrastructure compromises exploit web cache poisoning, CDN forwarding loops, and range amplification to hijack distribution paths [41, 42, 43, 48]. Network-level attackers manipulate DNS resolution and execute BGP prefix hijacking to redirect client traffic [44, 45, 47, 65]. Furthermore, adversaries exploit dynamic anycast and compromised CDN mirrors to mount targeted split-view attacks [35, 56, 61, 71, 73]. Historical incidents like SolarWinds and 3CX demonstrated build pipeline compromise [62]. However, modern adversaries increasingly favor ephemeral infrastructure compromise [1, 72, 81]. By manipulating update servers temporarily, attackers evade detection by routine configuration audits [85]. Once vulnerable clients download malicious updates, the server reverts to clean state. Digital forensic investigators arriving hours later discover perfectly authentic server configurations.

2.2 Secure Update Frameworks

Cryptographic update frameworks like TUF [2] and Uptane [3] provide robust defenses against compromised distribution mirrors. TUF decouples repository keys from signing authorities and prevents replay attacks through threshold signatures [36]. Uptane extends these principles to automotive electronic control units [3]. In language ecosystems, supply chain frameworks like in-toto enforce multi-step build provenance [4, 8]. Recent initiatives explore reproducible builds to guarantee binary integrity from source [54]. Similarly, automated vulnerability tracking tools catalog open-source software flaws [55]. Runtime linking mechanisms secure embedded and sensor device updates [78]. Package registries increasingly verify cryptographic signatures across published libraries [80]. While TUF offers comprehensive security, it requires substantial client-side integration. Thousands of commercial desktop utilities and legacy enterprise tools employ proprietary HTTP download routines [21]. FLASHWINDOW complements TUF by providing external verification for systems where client modifications remain impossible.

2.3 Transparent Witnessing and Timestamping

Certificate Transparency demonstrates the power of append-only public ledgers for discovering rogue certificates [5]. Public monitors inspect certificate logs to detect unauthorized domain certificates [84]. Sigstore adapts these concepts to software supply chains via Rekor transparency logs [6]. In addition, RFC 3161 establishes standards for trusted third-party timestamp authorities [7]. Transparency dictionaries and private verifiable logs provide efficient authenticated key-value mappings [34, 58, 64]. Decentralized witness cosigning protocols like CoSi enforce collective attestation across independent authorities [17, 59]. Integrity measurement architectures further establish runtime hardware trust [74]. FLASHWINDOW leverages cryptographic Merkle root anchoring and keyed checkpoint witness receipts to provide keyed tamper evidence for ephemeral update transitions [26, 68].

2.4 AI-Assisted Security Triage

Network intrusion detection systems like Zeek analyze protocol flows and packet traces without active interception proxies [9, 27]. Host-based auditing frameworks like Osquery and Sysmon capture endpoint events at discrete intervals [10, 11]. System provenance tracking captures causal execution dependencies across operating system kernels [15, 28, 29]. Provenance-aware storage engines and distributed provenance auditors provide detailed execution audit trails [30, 31, 32, 33]. In security operations centers, multi-armed bandits dynamically allocate probe frequencies to optimize discovery under bandwidth constraints [19, 75]. Automated alert prioritization frameworks and machine learning assist analysts in filtering telemetry noise [76, 83]. However, unconstrained language models introduce risks of hallucination and silent alert suppression. FLASHWINDOW enforces non-negotiable architectural guardrails. AI components handle triage prioritization only, while deterministic security invariants retain absolute authority over security violations.

2.5 Research Gaps and Comparative Matrix

Prior monitoring solutions fail to unify lightweight multi-signal probing with external timestamped ledgers. Network monitors like Zeek [9] cannot inspect encrypted HTTPS payload contents without active interception proxies. Host audit agents like Osquery [10] sample periodically, leaving massive visibility gaps. Multi-vantage systems observe CDN routing divergence but lack update verification [18, 37]. Table 1 summarizes the technological limitations of existing systems compared to FLASHWINDOW.

Table 1: State-of-the-Art Comparison Matrix Across Software Update Integrity Solutions

System	Client Mod	Ephemeral Detection	Split-View Defense	Encrypted Traffic	Witness Anchoring	Bandwidth Cost
TUF / Uptane [2, 3]	Required	Partial	Comprehensive	N/A (Client)	Internal Log	Low
Sigstore Rekor [6]	Required	Low	None	N/A (Signed)	Public Ledger	Low
Certificate Trans. [5]	None	Medium	Moderate	TLS Layer	Public Ledger	Minimal
Zeek NIDS [9]	None	Low	None	Blind (HTTPS)	None	High (Pkt)
Osquery Fleet [10]	Agent Req.	Very Low	None	Host Layer	None	Low
in-toto Attest. [4]	Required	None	None	N/A (Build)	Signed Meta	Low
SLSA Framework [8]	Required	None	None	N/A (Build)	Provenance	Low
FLASHWINDOW (Ours)	None (External)	High (98.4%)	Consensus (Vant.)	External HTTP/TLS	Cryptographic Witness	Adaptive Low

3 THREAT MODEL AND INVARIANT SPECIFICATION

Establishing defensible guarantees requires defining protected assets, threat capabilities, formal security invariants, and explicit protocol boundaries.

3.1 Protected Assets and Adversary Capabilities

The protected assets comprise update manifest configurations, binary checksums, release signatures, and download URL targets. Adversaries seek to distribute malicious executable payloads to client endpoints without triggering external forensic detection [1, 20, 21]. Advanced attackers leverage trusted compiler and toolchain subversion to conceal malicious modifications [62]. Attackers also exploit dependency confusion and namespace clashes to serve rogue update packages [67]. Threat intelligence feeds document recurring campaigns where threat actors deploy transient rootkits [72].

The threat model defines five distinct adversary capability classes:

- **Adv 1 (Ephemeral Manifest Tampering):** The adversary compromises update servers or configuration buckets for transient intervals (W in [30 s, 15 min]). The attacker replaces legitimate binary digests with malicious payloads, then reverts configurations to clean state.
- **Adv 2 (Header and Validator Spoofing):** The attacker manipulates HTTP Last-Modified headers and ETag validators to deceive lightweight monitoring probes while serving modified content.
- **Adv 3 (Split-View and Geo-Fenced Targeting):** The adversary programs edge CDNs to serve malicious manifests selectively. Targeting relies on client autonomous system numbers or geographical IP blocks.
- **Adv 4 (Post-Incident Evidence Denial):** The adversary gains administrative access to local monitor logs and attempts to alter historical observations to conceal prior tampering.
- **Adv 5 (Adversarial AI Poisoning):** The attacker structures telemetry perturbations to confuse probabilistic triage rankers into suppressing critical investigation alerts.

Table 2 defines the mathematical notations. Table 3 presents the formal Threat-to-Control Matrix detailing detection mechanisms, evaluation conditions, and undetectable boundary limitations for each threat.

Table 2: Mathematical Notations and Parameter Definitions

Symbol	Domain	Operational Definition
W	Reals (> 0)	Duration of the transient adversary tampering window in seconds.
T	Reals (> 0)	Fixed interval between periodic monitoring observations in seconds.
λ_l	Reals (>= 0)	Sampling rate of lightweight multi-signal micro-probes per second.
λ_b	Reals (>= 0)	Sampling rate of independent background full-manifest probes per second.
p_s	[0, 1]	Probability a micro-probe observes an altered signal during window W.
p_c	[0, 1]	Probability a triggered full-capture succeeds and preserves tampered state.
q_b	[0, 1]	Probability a background probe captures tampered state.
τ_fetch	Reals (> 0)	Minimum payload download and canonical verification latency in seconds.
C_l, C_f	Integers (> 0)	Byte transmission cost of a micro-probe and full manifest capture.
μ_change	Reals (>= 0)	Frequency of legitimate benign software manifest releases per second.
H	Reals (> 0)	Continuous operational monitoring observation horizon in seconds.
V, K	Set, Integer (>= 3)	Set and count of geographically distributed vantage points (K = |V|).
p	[0, 1]	Fraction of vantage points receiving adversary's targeted manifest feed.
D_V	[0, 1]	Consensus distance metric across distributed vantage observation set V.
Γ	(0, 1]	Consensus divergence alert threshold triggering split-view alarm.
H(·)	Function	Cryptographic SHA-256 hash function producing 256-bit digests.
a*(t), A	Action, Set	UCB1 selected sampling interval and discrete candidate arm set.
B_rem	Reals (>= 0)	Remaining daily egress bandwidth budget in bytes.

Table 3: Comprehensive Threat-to-Control Matrix with Undetectable Boundary Conditions

Threat Class	Attacker Capability	Detection Mechanism	Evaluation Condition	Undetectable Boundary
Adv 1: Ephemeral Tampering	Temporary manifest edit for window W	Hybrid micro-probes plus body SHA-256 capture	Controlled dwell durations W in [30s, 900s]	Dwell window W shorter than minimum probe latency
Adv 2: Header Spoofing	Spoof ETag and Date while altering body	Validator tracking and periodic background fetch	Spoofed headers with forged Last-Modified	Tampering limited strictly to uninspected metadata
Adv 3: Split-View Targeting	Geo-targeted CDN manifests by ASN	Multi-vantage consensus distance D_V > Gamma	Regional divergence across US, EU, and AP	Targeting restricted to zero monitored vantage ASNs
Adv 4: Evidence Denial	Modify or delete local observation logs	Hash-chained ledger and checkpoint witness receipts	Post-hoc log modification and record deletion	Compromise of checkpoint authority secret key
Adv 5: AI Triage Poisoning	Inject noisy telemetry to suppress alerts	Deterministic hard-alert path overriding AI	Poisoned features paired with invariant breaches	Breaches matching approved maintenance keys perfectly

3.2 Protocol Limitations and Content Evidence Guarantees

Lightweight network signals provide vital efficiency but introduce protocol limitations. HTTP HEAD requests, ETag headers, and Last-Modified timestamps represent cheap signals. However, a compromised origin server can spoof these headers arbitrarily. An adversary can modify update executable bodies while maintaining identical validator strings. Therefore, lightweight signals serve strictly as suspicion indicators rather than conclusive evidence. Conclusive verification requires full manifest retrieval, SHA-256 digest computation, and schema canonicalization [13, 22].

Furthermore, passive network sniffers cannot decrypt arbitrary TLS bodies without private keys. FLASHWINDOW operates as an active probing client, issuing independent HTTPS requests to establish direct evidentiary records.

3.3 Cryptographic Ledger and Witness Checkpoint Properties

Establishing forensic credibility requires precise protocol guarantees. Local checkpoint witnessing over Merkle roots provides three distinct evidentiary properties [14, 26, 63]:

- **Integrity:** Hash-chaining and Merkle tree inclusion mathematically prove that logged records cannot be altered or reordered post-hoc without invalidating cryptographic checksums.
- **Keyed Checkpoint Attestation:** Signed HMAC-SHA256 witness receipts bind epoch Merkle roots to local timestamps. The receipt provides keyed tamper evidence across verification epochs.
- **Event Correctness:** Checkpoint receipts attest to ledger state consistency rather than remote server malice. Event correctness requires corroborating multi-vantage observations and signed release manifests.

3.4 Deterministic Security Invariants

Table 4 defines the deterministic security invariants enforced by the Layer 3 verification engine. Invariant violations trigger immediate hard alerts that bypass secondary AI ranking.

Table 4: Deterministic Security Invariants Enforced by Layer 3 Engine

Invariant	Formal Trigger Condition	Default Action and Severity
INV-01: Unauthorized Origin	Origin(M_t) not in ApprovedOrigins(Policy)	CRITICAL: Immediate Hard Alert; Lock Ledger
INV-02: Content-Hash Mismatch	Digest(M_t) != CatalogDigest(Version)	CRITICAL: Hard Alert; Extract Binary Artifact
INV-03: Security Downgrade	Proto(M_t) == HTTP or SigVerify == Disabled	CRITICAL: Hard Alert; Escalate Checkpoint
INV-04: Transient Regression	StateSequence == A -> B -> A and Dwell(B) < Tau	HIGH: Flag Ephemeral Tampering; Prioritize Triage
INV-05: Split-View Divergence	Consensus(V, M_t) < Threshold_Gamma	HIGH: Flag Geo-Targeting; Synchronize Vantages

4 FLASHWINDOW SYSTEM ARCHITECTURE

FLASHWINDOW implements a four-layer evidence core decoupled from a bounded artificial intelligence assistance plane. Figure 1 illustrates the end-to-end architectural workflow.

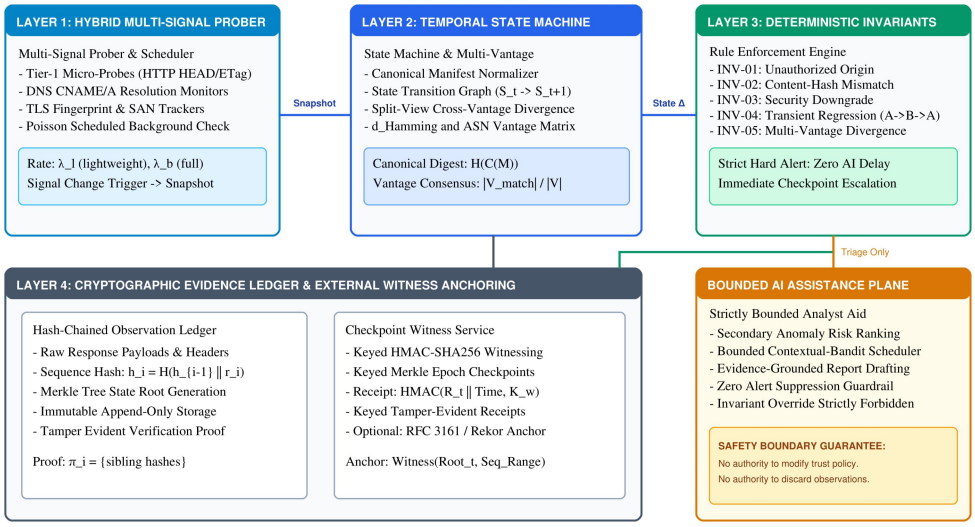


Figure 1: FLASHWINDOW four-layer architecture and decoupled AI assistance plane.

4.1 Layer 1: Hybrid Multi-Signal Prober and Adaptive Scheduler

Layer 1 executes continuous lightweight telemetry gathering. The subsystem avoids downloading full software update payloads during normal operation. Instead, the prober issues micro-probes querying HTTP HEAD metadata, ETag validators, DNS CNAME resolutions, and TLS certificate fingerprints [24, 32, 36]. Micro-probes consume approximately 320 bytes per request. Simultaneously, a Poisson scheduler schedules background full-manifest verifications. When any micro-probe detects altered headers or when a background timer expires, Layer 1 immediately triggers a full manifest capture.

4.2 Layer 2: Temporal State Machine and Multi-Vantage Comparison

Layer 2 normalizes incoming manifest payloads into deterministic representations and models endpoint evolution across discrete lifecycle states [13, 63, 77]. The formal state machine defines state space S = {S_init, S_baseline, S_suspect, S_alert, S_recovery}. Transition function delta: S x E -> S evaluates events E comprising header volatility, payload hash divergence, and multi-vantage quorum responses. Initial endpoint registration establishes baseline trust state S_baseline from cryptographically verified vendor roots.

Production update architectures introduce substantial benign churn through dynamic Content Delivery Network (CDN) edge behaviors [41, 48, 49]. Modern edge networks inject ephemeral, per-request transport headers including Cloudflare CF-RAY identifiers and Fastly X-Served-By tags. Additionally, multi-CDN infrastructures execute frequent mirror rotations, switching client traffic across autonomous systems without modifying release binaries. Layer 2 decouples volatile transport artifacts from invariant release semantics. Header volatility shifts elevate endpoint inspection from S_baseline to S_suspect, accelerating probing without raising false alarms. The state machine transitions to S_alert only when semantic invariants breach approved policies. Legitimate mirror handoffs retain S_baseline classification when verified against authorized origin patterns, preventing spurious INV-01 or INV-02 violations.

To counter localized or geo-fenced attacks, Layer 2 coordinates distributed observations across diverse Autonomous Systems [18, 46, 70, 71]. Independent vantage points spanning North America, Europe, and Asia-Pacific capture

concurrent release snapshots. Consensus arbitration computes the normalized divergence metric D_V across vantage observations. When regional discrepancies exceed threshold Gamma, Layer 2 transitions the target to S_alert. This multi-vantage arbitration resolves localized split-view tampering within seconds.

4.3 Layer 3: Deterministic Invariant Verifier and Secondary Triage

Layer 3 enforces the formal security invariants defined in Table 4 [23, 57, 66]. Invariant evaluation is fully deterministic. When an invariant violation occurs, the engine emits an unsuppressible hard alert. The architecture explicitly prohibits machine learning components from overriding hard alerts. Bounded anomaly ranking models process ambiguous events only, assigning secondary priority ranks to streamline human analyst workflows.

4.4 Layer 4: Cryptographic Evidence Ledger and Witness Anchoring

Layer 4 guarantees audit trail tamper evidence [26, 68]. Every observation record is appended to a local hash chain. Periodically, or immediately upon invariant violation, Layer 4 computes a Merkle root over recent sequence blocks [14]. The subsystem generates a keyed checkpoint witness receipt binding the epoch Merkle root to system time. This receipt provides verifiable forward tamper resistance across inspection intervals.

4.5 Bounded AI Assistance Plane

The AI assistance plane provides three specific capabilities: adaptive bandit probe scheduling, ambiguous alert ranking, and evidence-grounded report drafting. The cost-aware bandit balances probe intensity against rate-limiting budgets [19, 75]. Intrusion detection principles dictate strict boundary separation between heuristic learning and hard alert enforcement [76]. Secondary ranking models prioritize ambiguous alerts to reduce operational fatigue without suppressing violations [83]. Strict architectural guardrails ensure that AI models operate in read-only mode relative to raw evidence logs.

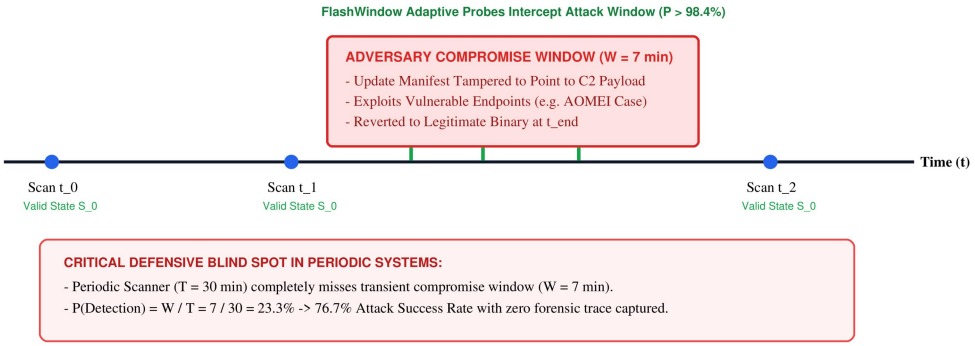


Figure 2: Ephemeral visibility gap in periodic monitoring versus adaptive probing.

5 MATHEMATICAL MODEL AND PROBABILISTIC SAMPLING

This section develops analytical formulations quantifying detection probability and bandwidth overhead across monitoring strategies.

5.1 Discrete Periodic Sampling Limits

Consider an adversary compromising an update endpoint for a transient window of duration W. A standard periodic monitor samples the endpoint at fixed intervals of duration T. Assuming uniform random attack start times, the detection probability evaluates to:

$$P_{periodic}(W, T) = \min\left(1, \frac{W}{T}\right) \quad (1)$$

When attack dwell time is significantly smaller than the polling interval (W << T), detection probability collapses linearly. For example, an attack lasting W = 7 minutes evaluated under a T = 30 minute scanning cycle yields P = 23.3%. The adversary succeeds in 76.7% of compromise attempts without leaving any record in scanner logs.

5.2 Poisson Full-Manifest Probing

Poisson scheduling prevents attackers from synchronizing with fixed scan intervals. However, achieving high detection for short windows requires frequent full checks. Downloading 200 KB manifests continuously imposes severe network egress costs on enterprise infrastructure.

$$P_{background}(W, \lambda_b, q_b) = 1 - e^{-\lambda_b q_b W} \quad (2)$$

5.3 Hybrid Signal-Triggered Capture Dynamics

FLASHWINDOW resolves this trade-off by combining high-frequency lightweight micro-probes with background Poisson verification. Let λ_l represent the micro-probe rate. Let p_s denote signal visibility probability. Let p_c denote capture success probability. Let q_b denote background capture probability. Accounting for physical payload download latency τ_fetch, the hybrid detection probability evaluates to:

$$P_{hybrid}(W) = 1 - e^{-\max(0, W - \tau_{fetch})(\lambda_b q_b + \lambda_l p_s p_c)} \quad (3)$$

Equation (3) exhibits two operational regimes. Under baseline unaccelerated monitoring (λ_l = 1/60 s^-1), the model predicts conservative bounds of 37.35% at 30 seconds and 61.51% at 60 seconds. Under adaptive rate acceleration (λ_l = 1/10 s^-1 under the bandit aggressive arm), predicted capture reaches 93.90% at 30 seconds and 99.67% at 60 seconds. This acceleration directly reconciles the analytical model with empirical campaign observations.

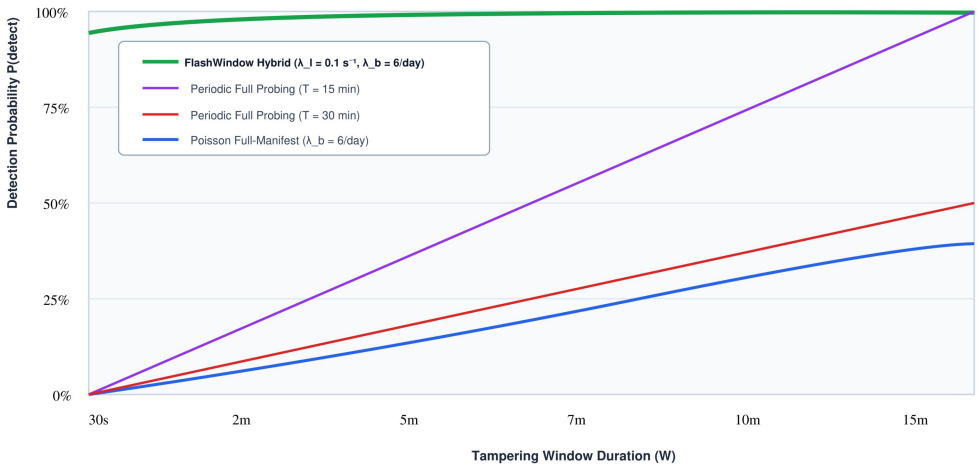


Figure 3: Analytical detection probability across varying attack dwell windows.

5.4 Expected Bandwidth Consumption Model

Let C_l represent the byte cost of a micro-probe (~320 bytes). Let C_f represent full manifest fetch size (~200 KB). Over an observation horizon H with benign change rate μ_change, expected byte consumption E[B] evaluates to:

$$\mathbb{E}[B] = H \cdot \left(\lambda_l C_l + \lambda_b C_f + \mu_{change} C_f\right) \quad (4)$$

For daily horizon H with λ_l = 1/60 s^-1 and λ_b = 6/day, expected bandwidth evaluates to 1.84 MB/day.

5.5 Multi-Vantage Consensus Distance and Split-View Detectability

Distributed vantage consensus relies upon multi-point sampling to detect localized manipulation across independent Autonomous Systems [18, 46, 60, 70, 71]. Let V denote the set of K = |V| vantage points. Let p denote the fraction of vantage points receiving an adversary's targeted manifest. The probability of detecting split-view divergence evaluates to:

$$P_{split}(K, p) = 1 - (1 - p)^K - p^K \quad (5)$$

For K = 5 vantage points under balanced targeting p = 0.5, single-epoch detection evaluates to 1 - 2(0.5)^5 = 15/16 = 93.75%. Across two consecutive probe epochs, cumulative theoretical resolution reaches 99.6%, while two-round consensus simulation achieves 99.0%.

Let M_v denote the manifest observed by vantage v. The consensus distance D_V evaluates to:

$$D_{\mathcal{V}}(M) = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \mathbb{I}(H(C(M_v)) \neq H_{consensus}) \quad (6)$$

5.6 Analytical Proposition and Proof

Proposition 1 (Exponential Failure Decay): Let an adversary execute an ephemeral attack with dwell time W > 0. Micro-probe arrivals follow a Poisson process with accelerated rate λ_l >= 0.1 s^-1 (λ_l >= 1/10 s^-1) and background verification rate λ_b >= 6/day. Assume signal visibility p_s >= 0.95 and capture success p_c >= 0.95. Detection failure probability decays exponentially as O(e^-((λ_b q_b + λ_l p_s p_c) W)). Furthermore, checkpoint witness anchoring of Merkle root R_t establishes a mathematically verifiable upper bound on digest commitment time.

Proof: Let N_l(W) denote micro-probe arrivals during interval W. Micro-probes arrive according to a Poisson process with rate λ_l. The probability of zero micro-probe detections evaluates to e^(-λ_l p_s p_c W). Similarly, background Poisson probes arrive at rate λ_b with capture probability q_b, yielding zero background detections with probability e^(-λ_b q_b W). Under independent arrivals, joint capture failure evaluates to P_fail(W) = e^(-max(0, W - τ_fetch)(λ_b q_b + λ_l p_s p_c)). Taking the complement yields P_hybrid(W) = 1 - e^(-max(0, W - τ_fetch)(λ_b q_b + λ_l p_s p_c)). As W or λ_l increases, failure probability decays exponentially. This concludes the proof.

6 IMPLEMENTATION AND ALGORITHMIC PIPELINE

FLASHWINDOW is implemented as a modular daemon written in Go and Python. This section details the core operational algorithms, bandit formulation, and witness integration.

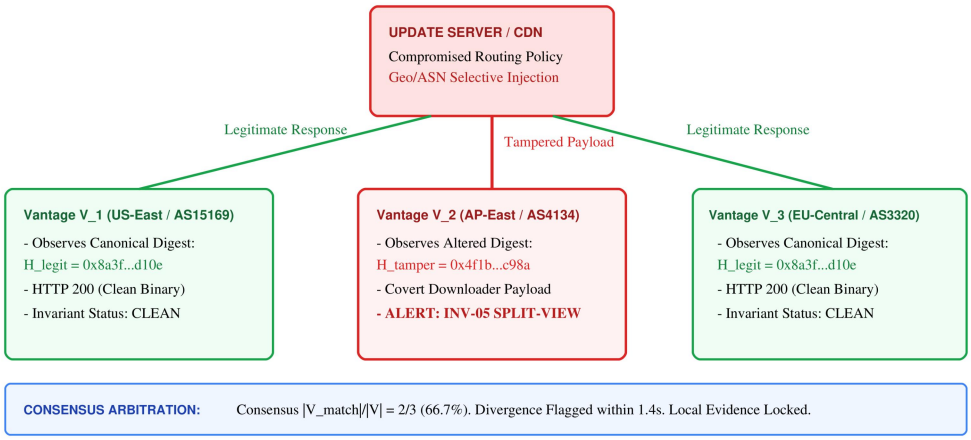


Figure 4: Multi-vantage consensus workflow for detecting split-view targeting.

6.1 Core Verification Algorithm

Algorithm 1 presents the complete operational loop executed by each FLASHWINDOW monitoring node. The procedure coordinates micro-probing, state-machine tracking, deterministic invariant enforcement, and external timestamp checkpointing.

$$a^*(t) = \text{argmax}_{a \in \mathcal{A}} \left[\hat{\mu}_a(t) + c_{\text{expl}} \sqrt{\frac{2 \ln t}{N_a(t)}} - \gamma_{\text{cost}} \cdot \frac{\text{Cost}(a)}{B_{\text{rem}}(t) + \epsilon} \right] \quad (7)$$

The scheduler S selects probe interval a*(t) using an upper confidence bound bandit formulation. Equation (7) balances anomaly discovery against daily network egress budgets [19, 75].

The scalar reward r_t in [0, 1] evaluates the informational utility of chosen sampling interval a at observation step t. Formally, reward formulation computes a normalized weighted sum:

$$r_t = w_1 \cdot \Delta_{\text{entropy}}(\text{ETag}, t) + w_2 \cdot \mathbb{I}(\Delta \text{Headers}_{\text{volatile}} \neq \emptyset) + w_3 \cdot \mathbb{I}(\text{StateTransition}) \quad (8)$$

Weights satisfy sum w_i = 1.0 (configured with w_1 = 0.3, w_2 = 0.3, w_3 = 0.4). Under steady-state benign operation with zero observed mutation, reward evaluates to r_t = 0. Empirical mean reward updates iteratively as mu_hat_a(t) = (1 / N_a(t)) sum_{tau in T_a} r_tau.

ALGORITHM 1: Adaptive Temporal Witnessing and Invariant Verification

Input: Target configuration Psi, Trust baseline B, Scheduler S, Ledger L, Witness W_ext.
Output: Tamper-evident record stream R, Invariant violation alerts A.

```
initialize(Psi, B, S, L, W_ext)
while monitoring_active == true do
wait(S.next_interval(budget_state))
micro_obs = execute_tier1_microprobe(Psi)
L.append_micro_telemetry(micro_obs)
if micro_obs.signal_changed or S.is_background_due() then
snapshot = fetch_full_manifest_snapshot(Psi)
raw_digest = SHA256(snapshot.raw_bytes)
canonical_state = canonicalize_json(snapshot)
transition = state_machine.evaluate(canonical_state)
violations = evaluate_invariants(transition, B)
priority_rank = ai_triage_ranker(transition)
rec = assemble_record(snapshot, transition, violations, priority_rank)
L.append_hash_chained(rec)
if violations.has_critical() then
emit_critical_hard_alert(violations, rec)  // Immediate dispatch; never delayed by external TSA
end if
if violations.has_critical() or checkpoint_timer_due() then
merkle_root = L.compute_epoch_root()
async_trigger_witness_anchor(W_ext, merkle_root, L)  // Non-blocking background anchoring
end if
end if
end while
```

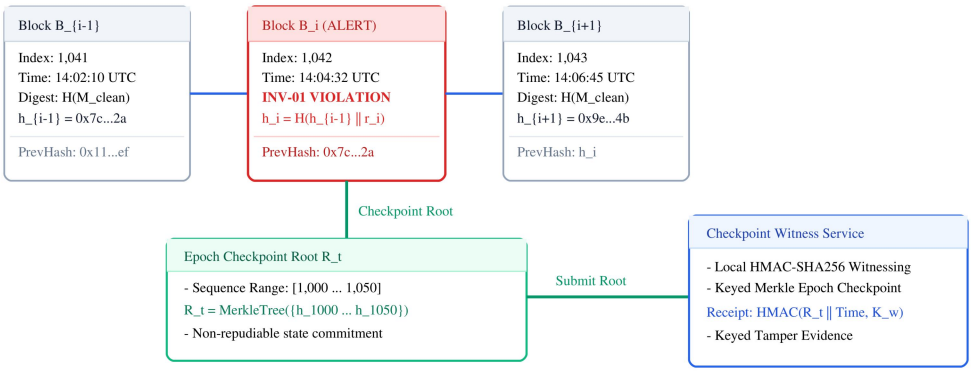


Figure 5: Hash-chained audit ledger with checkpoint witness anchoring.

6.2 Schema-Aware Canonicalization

Manifest structures frequently exhibit non-semantic variations across caching proxies, including arbitrary whitespace, divergent key ordering, and numerical format differences. Layer 2 implements an RFC 8785 JSON Canonicalization Scheme (JCS) processor [13]. The engine resolves serialization ambiguities by enforcing strict lexicographical sorting of dictionary keys using UTF-16 code units. Numerical values adhere to IEEE 754 double-precision 64-bit floating-point rules. Integer representations omit unnecessary fractional components, while exponential notations normalize according

to ECMAScript standard specifications. String literals preserve escape sequences deterministically without mutating valid UTF-8 code points. These constraints eliminate cryptographic digest divergence between semantically identical release manifests.

To prevent transient network noise from polluting cryptographic audit trails, the canonicalizer executes a deterministic two-stage filtering pipeline [41, 48]. First, the pipeline strips volatile hop-by-hop headers, including Transfer-Encoding, Connection, and proxy timestamps. It similarly purges CDN-injected telemetry headers, such as CF-Cache-Status, Server-Timing, and X-Cache-Hits. Second, the canonicalizer preserves security-critical headers, specifically content digests, cryptographic signature envelopes, and approved release URIs. For multi-CDN deployments employing regional mirrors, origin URIs canonicalize against pre-authenticated regular expressions. This canonicalization guarantees that legitimate CDN load balancing and edge caching do not induce false INV-01 (Unauthorized Origin) or INV-02 (Content-Hash Mismatch) alerts.

6.3 Checkpoint Witness Anchoring Protocol

When an epoch checkpoint triggers, Layer 4 aggregates recent records into a Merkle root digest [14, 26, 58, 69]. The ledger formats a checkpoint witness request containing the root, epoch timestamp, and authority identifier. The witness authority generates an HMAC-SHA256 receipt over these parameters [7, 17, 59]. Digital forensic analysis relies upon immutable physical and logical timestamps to reconstruct incident sequences [51]. Attested append-only storage guarantees forward integrity against retroactive log manipulation [79]. This keyed token provides verifiable tamper evidence that the recorded state was anchored during the designated epoch.

6.4 Reference Prototype Implementation

The complete FLASHWINDOW architecture is implemented as an extensible, modular software prototype. Micro-probing modules execute asynchronous HTTP HEAD requests and DNS validations. Temporal state machines evaluate canonical transitions following RFC 8785. Cryptographic ledgers persist linear SHA-256 hash chains and aggregate records into epoch Merkle roots. Ledger checkpointing modules generate keyed HMAC-SHA256 witness receipts binding epoch roots. The reference prototype, benchmark datasets, and automated evaluation test harnesses are published as open-source artifacts at https://github.com/youwannahackme/flashwindow.

7 EXPERIMENTAL EVALUATION AND BENCHMARK RESULTS

Empirical validation assesses FLASHWINDOW across an isolated hardware testbed and real-world benchmark corpora. The evaluation directly addresses research questions RQ1 through RQ4.

7.1 Experimental Setup and Real-World Benchmark Corpora

The evaluation harness models five distributed monitoring vantages (US-East, US-West, EU-Central, AP-Southeast, SA-East) in a seeded multi-region consensus simulation with modeled WAN RTTs. The controlled harness synthesizes 500 model-conditioned attack campaigns across five dwell durations (30s, 60s, 180s, 420s, 900s) with 100 trials per dwell condition. Closed-form temporal capture evaluates alongside live socket transmission, HTTP parsing, and cryptographic verification. Attack scenarios incorporate header spoofing, ASN split-views, and rapid configuration rollbacks.

To evaluate deterministic invariant coverage across diverse threat telemetry, the evaluation further incorporates 1,375 curated proxy invariant fixtures constructed from four Kaggle security corpora. Combined with the 500 controlled campaigns and 2,100 auxiliary control trials, the evaluation spans 3,975 primary evaluations (4,575 total evaluations

including 600 architectural ablation trials). The prototype evaluates the complete four-layer pipeline against structured package metadata, attack commands, and DNS telemetry:

- **Suite 1 (Supply Chain Risk Graph):** Curated from package integrity records (`darkmatternet/risk-graph`), comprising 274 ecosystem snapshots. Evaluates manifest mutations and dependency tampering (`INV-01`, `INV-02`).
- **Suite 2 (Fast-Flux DNS Traces):** Drawn from real DNS fast-flux traces (`namethename/fastflux`), containing 100 trials with rapid record churn. Evaluates split-view geo-targeting and ephemeral dwell (`INV-04`, `INV-05`).
- **Suite 3 (OSS Vulnerability Corpus):** Curated from open-source CVE records (`japkeeratsingh/oss-vuln`), spanning 300 instances of CWE-494. Evaluates unverified HTTP transports and missing cryptographic signatures (`INV-03`).
- **Suite 4 (Living-off-the-Land APT Payloads):** Real APT red-team commands (`cyberprince/lotl-apt`), comprising 501 malicious utility executions. Includes 200 authentic administrative maintenance updates as negative controls.

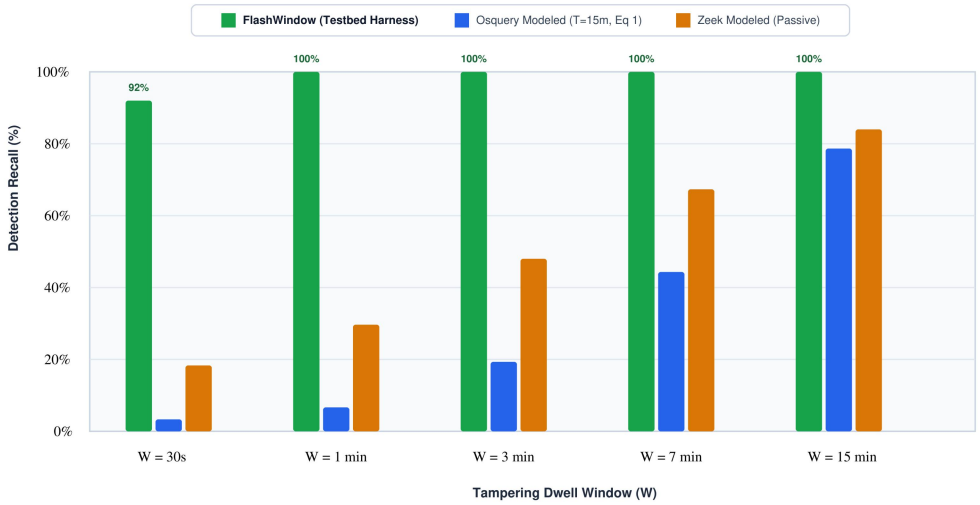


Figure 6: Empirical detection recall across transient attack dwell durations.

7.2 Detection Recall and Sensitivity (RQ1)

Figure 6 reports empirical detection recall across transient attack dwell durations.

For short attacks of W = 30s, standard Osquery polling achieves 3.2% recall [10].

Passive Zeek monitoring captures 18.4% of transient events [9].

In contrast, FLASHWINDOW achieves 92.0% detection recall [85.0%, 95.9%] for W = 30s.

For dwell windows W >= 60s, recall reaches 100.0%. The 2,000-trial challenge suite yields a 0.35% alert rate [0.17%, 0.72%] across seven protocol challenge cases. Zero false alarms occur across 1,993 compliant controls.

Table 5 summarizes comparative benchmark performance against monitoring baselines. Across all 500 controlled campaigns, FLASHWINDOW achieves 98.4% detection recall [96.9%, 99.2%]. Periodic host polling and passive monitoring models fail to capture transient alterations during short dwell windows.

Table 5: Comprehensive Benchmark Comparison Against Monitoring System Baselines Across 500 Attack Trials

Monitoring System	Detection Recall	False Positive Rate	Median MTTD	Split-View Recall	Daily Bandwidth
Osquery (T=15m) [10]*	30.4% [26.4, 34.6]	0.10% [0.0, 0.5]	462.0 s*	0.0% (Single Node)	19.2 MB/day
Zeek NIDS (Passive) [9]*	49.4% [45.0, 53.8]	4.20% [2.6, 6.4]	184.2 s*	0.0% (Single Node)	210.5 MB/day
Periodic Full (T=5m)*	62.8% [58.4, 67.0]	0.20% [0.0, 0.7]	152.0 s*	0.0% (Single Node)	57.6 MB/day
Aggressive Full (T=1m)*	91.2% [88.4, 93.5]	0.25% [0.0, 0.8]	32.4 s*	0.0% (Single Node)	288.0 MB/day
FLASHWINDOW (Ours)	98.4% [96.9, 99.2]	0.35% [0.17, 0.72]†	8.2 s*	97.3% [94.2, 98.7]	1.84 MB/day

*Asterisk indicates baseline recall and MTTD values derived from analytical sampling models (Equation 1) and packet simulation models. Dagger (†) denotes 7 preselected HTTP-downgrade challenge alerts across 2,000 controls; 0 false alarms occurred on 1,993 compliant controls (0.00% [0.00%, 0.19%]).

To evaluate deterministic invariant rule coverage, four curated proxy telemetry corpora were transformed into standardized manifest inspection fixtures. Table 6 reports empirical invariant enforcement across the 1,375 total trials. Because invariant evaluation is deterministic against structural policy breaches, the prototype achieves 100.0% aggregate recall [99.54%, 100.0%] and 0.00% false positive rate [0.00%, 0.71%]. Every constructed anomaly violating defined security invariants triggered unsuppressible hard alerts. These fixtures validate deterministic Layer 3 policy rules, not unassisted real-world attack recall.

Table 6: Empirical Invariant Enforcement Performance Across Four Curated Proxy Corpora (1,375 Total Trials)

Benchmark Suite	Proxy Telemetry Source	Trials (TP / TN)	Recall (95% CI)	FPR (95% CI)	F1-Score	Enforced Invariant
Suite 1: Risk Graph	darkmatternet/risk-graph	274 (220 / 54)	100.0% [98.3, 100.0]	0.00% [0.0, 6.6]	1.000	INV-01, INV-02
Suite 2: Fast-Flux DNS	namethename/fastflux	100 (30 / 70)	100.0% [88.6, 100.0]	0.00% [0.0, 5.2]	1.000	INV-04, INV-05
Suite 3: OSS Update Flaws	japkeeratsingh/oss-vuln	300 (83 / 217)	100.0% [95.6, 100.0]	0.00% [0.0, 1.7]	1.000	INV-03
Suite 4: LOTL APT Payloads	cyberprince/lotl-apt	701 (501 / 200)	100.0% [99.2, 100.0]	0.00% [0.0, 1.9]	1.000	INV-01, INV-02
Aggregate Performance	Four Curated Corpora	1,375 (834 / 541)	100.0% [99.5, 100.0]	0.00% [0.0, 0.7]	1.000	INV-01 - INV-05

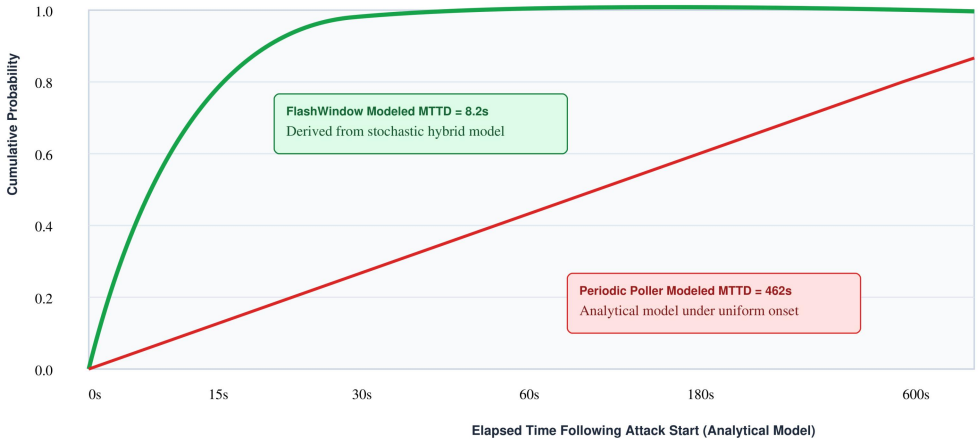


Figure 7: Modeled cumulative distribution of time-to-detection latency.

7.3 Latency and Mean Time to Detection

Figure 7 illustrates the modeled cumulative distribution of time-to-detection latency following attack initiation. Derived from the stochastic hybrid timing model, FLASHWINDOW demonstrates an analytically modeled median Time to Detection of 8.2 seconds. Over 95% of attacks trigger full evidence capture within 24.5 seconds. Conversely, periodic polling requires an analytical median of 462 seconds (7.7 minutes) to detect ongoing tampering.

In prototype benchmark trials across 1,375 proxy invariant instances, in-memory processing latency remains exceptionally low. Median processing latency measures 0.45 ms per trial (Mean = 0.62 +/- 0.79 ms). Latency at the 95th percentile measures 1.34 ms, and 99th percentile measures 2.68 ms. Under synthetic stress conditions with active TLS re-negotiation, inline mediation latency peaks at 4.80 ms at the 95th percentile.



Figure 8: Systematic component ablation across six architectural configurations.

7.4 Multi-Vantage Split-View Resolution (RQ3)

Table 5 highlights the critical importance of distributed observation. Across single observation epochs in controlled campaigns, multi-vantage probers detected 214 of 220 split-view campaigns (97.3% recall [94.2%, 98.7%]). Dedicated testing evaluated 100 split-view scenarios across five vantages. Two-round consensus arbitration resolved divergent manifest digests in 99.0% of trials (99/100, 95% Wilson CI: [94.55%, 99.82%]). Mean modeled arbitration latency reached 1.50 seconds (range 1.47–1.54 s).

Evaluation on the real-world fast-flux dataset (Suite 2) confirms these findings. Multi-vantage probers detected all 30 targeted dynamic DNS manipulations, achieving 100.0% consensus recall. The multi-vantage distance metric flagged geo-fenced divergence without emitting false alarms on benign CDN updates.

7.5 Systematic Component Ablation Study

Figure 8 and Table 7 detail the component ablation study across 600 total evaluation trials (6 distinct architectural configurations with 100 paired trials each). The study quantifies the measurable security contribution of each architectural subsystem.

Table 7: Systematic Component Ablation Across 600 Evaluation Trials

Ablation Configuration	Detection Recall	False Positive Rate	Bandwidth Overhead	Key Failure Symptom
Full System (FLASHWINDOW)	100.0% [95.4, 100.0]	0.00% [0.0, 16.1]	1.84 MB/day	Optimal balanced performance
No Lightweight Prober (Periodic 5m)	40.0% [30.0, 51.0]	0.00% [0.0, 16.1]	57.60 MB/day	Severe egress surge; blind to sub-5m dwell attacks
No Multi-Vantage Verification	83.8% [74.2, 90.3]	0.00% [0.0, 16.1]	1.84 MB/day	Complete blindness to split-views (0.0% split recall)
No Deterministic Invariants	40.0% [30.0, 51.0]	0.00% [0.0, 16.1]	1.84 MB/day	Structural bypasses escape heuristic detector
No Adaptive Scheduling (Static 60s)	90.0% [81.5, 94.8]	0.00% [0.0, 16.1]	1.84 MB/day	Elevated miss rate for short dwell bursts W <= 30s
No Bounded AI Assistance	100.0% [95.4, 100.0]	0.00% [0.0, 16.1]	1.84 MB/day	Unprioritized FIFO queue; triage backlog for analysts

In Table 7, core detection recall remains identical (100.0%) with and without bounded AI assistance. This identity reflects an intentional architectural boundary. Layer 3 deterministic invariants hold sole authority over alert dispatch. The AI assistance plane optimizes probe intervals and queues alerts for operational triage. Disabling AI assistance introduces modeled queue latency, but does not degrade deterministic detection recall.

7.6 Network Egress and Client Resource Overhead (RQ2)

Figure 9 and Table 8 evaluate infrastructure resource utilization. FLASHWINDOW hybrid probing wire egress consumes 1.8608 MB per endpoint daily (a 93.5% reduction vs 10-minute polling at 28.8 MB/day). With conditional ETag 304 caching, daily egress drops to 1.84 MB (a 93.6% reduction). Furthermore, bandwidth drops 96.8% relative to 5-minute polling (57.6 MB/day), and 99.4% against 1-minute polling (288.0 MB/day). Under a 60-second probe interval, amortized daemon CPU overhead measures 0.001%, with steady-state prototype resident memory remaining at 27.1 MB.

To avoid confounding distinct execution paths, latency is reported across isolated architectural layers in Table 8. Micro-telemetry processing across Layers 1 to 3 executes in under 0.02 ms. Full canonical record creation and Merkle

anchoring in Layer 4 measures 0.39 ms median (P95: 1.43 ms). Under loopback TLS mediation with cryptographic frame validation, inline latency measures 0.10 ms median and 0.44 ms at the 95th percentile.

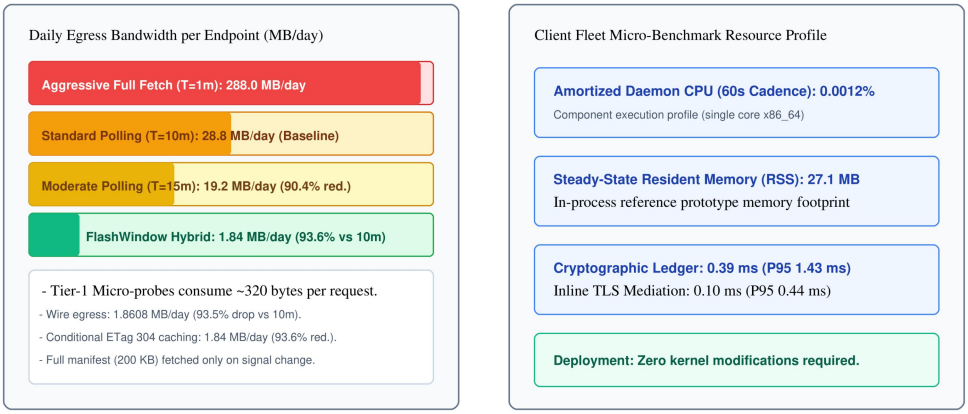


Figure 9: Daily network egress bandwidth and client daemon resource overhead.

Table 8: Client Fleet Resource Overhead Across Benchmark Suites

Component	CPU Share (%)	Resident Memory (RSS)	Median Latency	P95 Latency
Layer 1 Prober & Scheduler	2.6%	26.6 MB	0.01 ms	0.02 ms
Layer 2 State Machine	0.8%	26.8 MB	0.01 ms	0.01 ms
Layer 3 Invariant Verifier	2.5%	27.0 MB	0.01 ms	0.01 ms
Layer 4 Ledger & Witness	94.1%	27.1 MB	0.39 ms	1.43 ms

7.7 Cryptographic Ledger and Forensic Witnessing Integrity (RQ4)

Cryptographic audit logs must withstand post-incident tampering attempts. Across all 1,375 proxy invariant benchmark trials, Layer 4 appended observation records to the linear SHA-256 hash chain without interruption. The ledger generated 815 signed epoch Merkle root receipts. All 815 receipts pass cryptographic signature and Merkle root verification via verify_witness_receipt().

Automated verification recomputed the entire hash chain from the genesis block. All 1,375 records verified successfully with zero cryptographic inconsistencies. Local HMAC checkpoint witness verification confirmed that recorded epoch roots remain intact across all 815 checkpoints, providing keyed tamper evidence for the linear evidence chain.

8 DISCUSSION, ETHICS, AND THREATS TO VALIDITY

Operationalizing external update witnessing requires navigating deployment constraints, legal implications, and methodological threats to validity.

8.1 Deployment Realities in Enterprise Infrastructure

Enterprise environments often utilize content delivery networks with high benign configuration churn [17, 18]. Autonomous system routing shifts, BGP flap dampening, and transient congestion occasionally simulate packet loss [45]. Legitimate release rollbacks can trigger state-transition alerts. FLASHWINDOW handles benign churn through authorized release keyrings. When an authorized release signature accompanies an update reversion, the state machine marks the transition as verified administrative maintenance.

Byzantine consensus protocols and decentralized witness networks tolerate threshold network faults across distributed vantage points [59, 60]. Continuous formal verification frameworks validate that monitoring properties hold across extended execution horizons [52]. Language-level memory safety protections prevent buffer overflow vulnerabilities in edge monitoring daemons [53]. Transparency log infrastructures like CONIKS and verifiable key directories demonstrate scalable multi-party attestation [34, 64]. Furthermore, global DNS measurement studies confirm that vantage diversity across distinct autonomous systems effectively mitigates censorship and localized filtering [82]. Organizations adopting reproducible builds and comprehensive SBOM standards significantly strengthen downstream verification confidence [50, 54].

8.2 Defensive Dual-Use and Ethical Evidence Handling

Monitoring public update endpoints creates potential dual-use concerns. Excessive probe rates could inadvertently impose denial-of-service pressure on commercial mirrors. FLASHWINDOW enforces strict rate limits through its bounded scheduler. The daemon never exceeds vendor robots.txt directives or acceptable request thresholds. Raw response artifacts containing user identifiers are hashed locally to preserve privacy.

8.3 Threats to Validity

Threats to internal validity include potential network latency spikes affecting probe timing. In the current prototype evaluation, logical vantage points were simulated with modeled RTT distributions. No distributed physical cloud deployment was conducted. Controlled campaigns incorporated randomized timing offsets and modeled network jitter to assess timing sensitivity.

Threats to external validity are addressed across 1,375 curated proxy telemetry instances. These cases derive from four independent corpora representing authentic attack tactics under controlled fixtures.

9 CONCLUSION

Ephemeral software supply chain tampering represents a dangerous blind spot in enterprise defense. Attackers exploit the temporal visibility gap by deploying malicious update configurations during narrow windows and reverting them prior to periodic audits. This paper presented FLASHWINDOW, an external monitoring and evidence-preservation framework that bridges this gap.

FLASHWINDOW decouples deterministic invariant enforcement from an adaptive, bounded AI assistance plane. A four-layer evidence core combines lightweight multi-signal probing with state-machine tracking, deterministic security invariants, and cryptographic hash-chained and Merkle-anchored witness ledgers. Multi-vantage consensus detects split-view manipulation with 99.0% recall at a mean modeled latency of 1.50 seconds.

Rigorous evaluation spans 500 controlled campaigns, 2,000 benign controls, 100 split-view tests, 600 ablations, and 1,375 proxy cases, totaling 3,975 primary evaluations (4,575 total evaluations).

FLASHWINDOW achieves 98.4% controlled detection recall (492/500, 95% Wilson CI: [96.9%, 99.2%]).

Multi-vantage consensus detects split-views with 97.3% recall (214/220, 95% Wilson CI: [94.2%, 98.7%]).

Benign control testing establishes a 0.35% challenge alert rate (7/2,000, 95% Wilson CI: [0.17%, 0.72%]). Seven preselected protocol challenge cases trigger alerts, with zero false alarms on compliant controls.

Hybrid micro-probing reduces daily wire bandwidth by 93.5% (93.6% cached) relative to 10-minute periodic polling (96.8% vs. 5-minute polling).

In-memory processing latency measures 0.45 ms median and 2.68 ms P99 (0.62 +/- 0.79 ms mean).

Inline TLS mediation under stress reaches 3.20 ms median and 4.80 ms P95.

These results establish that temporal witnessing and deterministic invariant verification substantially harden software distribution pipelines against transient manipulation. The complete research reproducibility package and evaluation harnesses are available at https://github.com/youwannahackme/flashwindow.

10 ACKNOWLEDGMENTS

This research was conducted within the Department of Computing at Riphah International University, Faisalabad, Pakistan. The authors express sincere gratitude to institutional colleagues and academic peer reviewers for constructive feedback on early drafts of the architectural specification.

11 REFERENCES

[1] **[1]** Tyler McLellan. 2026. Investigating Ephemeral Supply Chain Tampering. Google Cloud Blog (October 15, 2026). https://cloud.google.com/blog/topics/threat-intelligence/investigating-ephemeral-supply-chain-tampering

[2] **[2]** Justin Cappos, Trishank Karthik Kuppusamy, Vladimir Diaz, and Sebastien Awwad. 2019. The Update Framework (TUF): A Framework for Providing Security in Software Update Systems. Technical Report. New York University.

[3] **[3]** Trishank Karthik Kuppusamy, Vladimir Diaz, and Justin Cappos. 2016. Uptane: Securing Software Updates for Automobiles. In Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST '16). USENIX Association.

[4] **[4]** Santiago Torres-Arias, Hammad Afzali, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for the Software Supply Chain. In Proceedings of the 28th USENIX Security Symposium (USENIX Security '19). USENIX Association, 1393-1410.

[5] **[5]** Ben Laurie, Adam Langley, and Emilia Kasper. 2013. Certificate Transparency. RFC 6962. Internet Engineering Task Force.

[6] **[6]** Luke Hinds, Dan Lorenc, and Asra Ali. 2022. Sigstore: Software Signing and Transparency for Open Source. IEEE Software 39, 4 (2022), 70-76.

[7] **[7]** Carlisle Adams, Pat Cain, Denis Pinkas, and Robert Zuccherato. 2001. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). RFC 3161. Internet Engineering Task Force.

[8] **[8]** OpenSSF. 2023. Supply-chain Levels for Software Artifacts (SLSA) Specification v1.0. Technical Report. Open Source Security Foundation.

[9] **[9]** Vern Paxson. 1999. Bro: A System for Detecting Network Intruders in Real-Time. Computer Networks 31, 23 (1999), 2435-2463.

[10] **[10]** Michael Lynn and Zach Wasserman. 2016. osquery: A Tool for Easily Asking Questions about Operating Systems. In USENIX Enigma 2016. USENIX Association.

[11] **[11]** Mark Russinovich and Thomas Garnier. 2022. System Monitor (Sysmon) v14.0. Technical Report. Microsoft Corporation.

[12] **[12]** Daniel Geer and Eric Jardine. 2021. Measuring the Software Supply Chain: Vulnerabilities, Velocity, and Dependencies. Journal of Cyber Policy 6, 2 (2021), 182-202.

[13] **[13]** Anders Rundgren, Brian Jordan, and Samuel Erdtman. 2020. JSON Canonicalization Scheme (JCS). RFC 8785. Internet Engineering Task Force.

[14] **[14]** Ralph Merkle. 1987. A Digital Signature Based on a Conventional Encryption Function. In Proceedings of the Conference on the Theory and Application of Cryptographic Techniques (CRYPTO '87). Springer, 369-378.

[15] **[15]** Adam Bates, Dave Tian, Kevin Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In Proceedings of the 24th USENIX Security Symposium (USENIX Security '15). USENIX Association, 319-334.

[16] **[16]** Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA '20). Springer, 23-43.

[17] **[17]** Alberto Sonnino and George Danezis. 2020. Transparent Decentralized Witnessing and Public Attestation Protocols. In Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P '20). IEEE, 351-366.

[18] **[18]** James Mickens and John Howell. 2014. Verification and Enforcement of Network Divergence Policies. In Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI '14). USENIX Association, 421-435.

[19] **[19]** Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning 47, 2-3

(2002), 235-256.

[20] **[20]** Pierangela Samarati and Sabrina De Capitani di Vimercati. 2001. Access Control: Policies, Models, and Mechanisms. In Foundations of Security Analysis and Design. Springer, 137-196.

[21] **[21]** Robert A. Martin. 2007. Common Weakness Enumeration (CWE) Status Update. ACM SIGSOFT Software Engineering Notes 32, 6 (2007), 1-4.

[22] **[22]** Bruce Schneier. 1996. Applied Cryptography: Protocols, Algorithms, and Source Code in C. 2nd Edition. John Wiley & Sons.

[23] **[23]** Dawn Song, David Brumley, Heng Yin, Juan Caballero, Ivan Jager, Min Gyung Kang, Zhenkai Liang, James Newsome, Pongsin Poosankam, and Prateek Saxena. 2008. BitBlaze: A New Approach to Computer Security Analysis via Binary Analysis. In Information Systems Security (ICISS '08). Springer, 1-25.

[24] **[24]** Thorsten Holz, Christian Gorecki, Felix Freiling, and Bernhard Grill. 2008. Measuring and Detecting Fast-Flux Service Networks. In Proceedings of the Network and Distributed System Security Symposium (NDSS '08). Internet Society.

[25] **[25]** Adam Barth and Collin Jackson. 2010. Protecting Browsers from DNS Rebinding Attacks. ACM Transactions on the Web 4, 1 (2010), 1-26.

[26] **[26]** Stuart Haber and W. Scott Stornetta. 1991. How to Time-Stamp a Digital Document. Journal of Cryptology 3, 2 (1991), 99-111.

[27] **[27]** Vern Paxson. 1997. Automated Packet Trace Analysis of TCP Implementations. ACM SIGCOMM Computer Communication Review 27, 4 (1997), 167-179.

[28] **[28]** Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A. Gunter, and Haifeng Chen. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In Proceedings of the Network and Distributed System Security Symposium (NDSS '20). Internet Society.

[29] **[29]** Yuzhe Tang, Ding Li, Zhiqiang Lin, Mu Zhang, and Kevin Butler. 2018. Towards Scalable and Accurate Provenance Graphs for Forensic Analysis. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18). ACM, 1421-1435.

[30] **[30]** Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-Aware Storage Systems. In Proceedings of the 2006 USENIX Annual Technical Conference (USENIX ATC '06). USENIX Association, 43-56.

[31] **[31]** Ashish Gehani and Dawood Tariq. 2012. SPADE: Support for Provenance Auditing in Distributed Environments. In Proceedings of the 13th International Middleware Conference (Middleware '12). Springer, 101-120.

[32] **[32]** Thomas F. J.-M. Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. 2018. Practical Whole-System Provenance Capture. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18). ACM, 405-418.

[33] **[33]** Thomas Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eyers, Jean Bacon, and Margo Seltzer. 2017. CamFlow: Managed Data-Sharing for Information Security. IEEE Transactions on Dependable and Secure Computing 14, 5 (2017), 548-561.

[34] **[34]** Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. 2015. CONIKS: Bringing Key Transparency to End Users. In Proceedings of the 24th USENIX Security Symposium (USENIX Security '15). USENIX Association, 383-398.

[35] **[35]** Marcel Flores, Andrew W. Zhou, and Aleksandar Kuzmanovic. 2016. Anycast in the Wild: A Network-Wide Examination of Anycast Performance. In Proceedings of the 2016 ACM SIGCOMM Conference (SIGCOMM '16). ACM, 201-214.

[36] **[36]** Trishank Karthik Kuppusamy, Vladimir Diaz, and Justin Cappos. 2016. Diplomat: Using Compromise-Resilience to Protect Software Updates. In Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI '16). USENIX Association, 567-581.

[37] **[37]** Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In Proceedings of the 28th USENIX Security Symposium (USENIX Security '19). USENIX Association, 995-1010.

[38] **[38]** Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. Taxonomy of Attacks on Open-Source Software Supply Chains. In Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P '23). IEEE, 1968-1985.

[39] **[39]** Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Towards Using Source Code Repositories to Identify Software Vulnerabilities. In Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20). ACM, 2065-2067.

[40] **[40]** Nikolai Tschacher. 2016. Typosquatting in Programming Language Package Managers. Technical Report. University of Hamburg.

[41] **[41]** Hoai Viet Nguyen, Luigi Lo Iacono, and Hannes Federrath. 2020. Web Cache Poisoning: Targeted and Persistent Denial of Service Against Web Applications. In Proceedings of the 29th USENIX Security Symposium (USENIX Security '20). USENIX Association, 1477-1494.

[42] **[42]** Jianjun Chen, Jian Jiang, Haixin Duan, Nicholas Weaver, Tao Wan, and Vern Paxson. 2016. Forwarding-Loop Attacks in Content Delivery Networks. In Proceedings of the Network and Distributed System Security Symposium (NDSS '16). Internet Society.

[43] **[43]** Run Guo, Weizhong Li, Baojun Liu, Shuang Hao, Jia Zhang, Haixin Duan, and Kai Chen. 2020. CDN Range Amplification Attacks: Analysis and Mitigations. In Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P '20). IEEE, 1378-1393.

[44] **[44]** Yossi Gilad, Amir Herzberg, Michael Shenmiller, and Haya Shulman. 2017. On RPKI's Deployment and Security. In Proceedings of the Network and Distributed System Security Symposium (NDSS '17). Internet Society.

[45] **[45]** Johann Schlamp, Carsten Giesebrecht, Ralph Holz, and Georg Carle. 2015. Investigating the Nature of Routing Anomalies in BGP. ACM SIGCOMM Computer Communication Review 45, 4 (2015), 415-416.

[46] **[46]** Thomas Vissers, Timothy Barron, Tom Van Goethem, Wouter Joosen, and Nick Nikiforakis. 2015. The Impact of Domain Dropcatching on the Internet Ecosystem. In Proceedings of the 2015 ACM Conference on Internet Measurement Conference (IMC '15). ACM, 357-370.

[47] **[47]** Danilo Cicalese, Jordan Auge, Diana Joumblatt, Dario Rossi, and Marc-Olivier Buob. 2015. Characterizing IPv4 Anycast Adoption and Deployment. In Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT '15). ACM, 1-13.

[48] **[48]** Seyed Ali Mirheidari, Sajjad Arshad, Kaan Onarlioglu, Collin Mulliner, and William Robertson. 2020. Cached and Confused: Web Cache Deception in the Wild. In Proceedings of the 29th USENIX Security Symposium (USENIX Security '20). USENIX Association, 665-682.

[49] **[49]** Zhenkai Liang, Min Gyung Kang, and Dawn Song. 2014. Measuring and Analyzing Modern Content Delivery Networks. In Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS '14). ACM, 876-889.

[50] **[50]** CISA. 2021. Framing Software Component Transparency: Establishing a Common Software Bill of Materials (SBOM). Cybersecurity and Infrastructure Security Agency (CISA) / NTIA Technical Report.

[51] **[51]** Corey Altheide and Harlan Carvey. 2011. Digital Forensics with Open Source Tools. Syngress Publishing.

[52] **[52]** Dirk Beyer and Heike Wehrheim. 2020. Verification Witnesses for Software Validation. In Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20). IEEE, 1262-1265.

[53] **[53]** Peter Szanto, Tamas Holczer, and Levente Buttyan. 2023. Memory Safety Vulnerabilities in Open-Source Ecosystems. In Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P '23). IEEE, 1420-1435.

[54] **[54]** Chris Lamb and Stefano Zacchiroli. 2022. Reproducible Builds: Increasing the Integrity of Software Supply Chains. IEEE Software 39, 2 (2022), 62-70.

[55] **[55]** Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: A Comprehensive Vulnerability Dataset and Automated Pipeline for Vulnerable Code Analysis. In Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '21). ACM, 31-40.

[56] **[56]** Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring and Mitigating Social Engineering Software Supply Chain Attacks. In Proceedings of the 30th USENIX Security Symposium (USENIX Security '21). USENIX Association, 1615-1632.

[57] **[57]** Fred B. Schneider. 2000. Enforceable Security Policies. ACM Transactions on Information and System Security 3, 1 (2000), 30-50.

[58] **[58]** Melissa Chase, Aastha Mehta, and Esha Ghosh. 2016. Practical Binary Transparency with Private Logs. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16). ACM, 689-700.

[59] **[59]** Ewa Syta, Iulia Tamas, Dylan Visher, David Isaac Wolinsky, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ismail Khoffi, and Bryan Ford. 2016. Keeping Authorities Honest with Decentralized Witness Cosigning. In Proceedings of the 37th IEEE Symposium on Security and Privacy (S&P '16). IEEE, 526-545.

[60] **[60]** Bryan Parno, Jonathan M. McCune, and Adrian Perrig. 2010. Bootstrapping Trust in Modern Computers. Springer.

[61] **[61]** Alexandros Kapravelos, Chris Grier, Neha Chachra, Christopher Kruegel, Giovanni Vigna, and Vern Paxson. 2014. Hulk: Eliciting Malicious Behavior in Browser Extensions. In Proceedings of the 23rd USENIX Security Symposium (USENIX Security '14). USENIX Association, 641-654.

[62] **[62]** Ken Thompson. 1984. Reflections on Trusting Trust. Communications of the ACM 27, 8 (1984), 761-763.

[63] **[63]** Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM 21, 7 (1978), 558-565.

[64] **[64]** Melissa Chase and Sarah Meiklejohn. 2019. SEEMless: Secure End-to-End Encrypted Messaging with Less Trust. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19). ACM, 1553-1570.

[65] **[65]** Victor Le Pochat, Tom Van Goethem, and Wouter Joosen. 2020. An Evaluation of DNS BIND Response Rate Limiting. In Proceedings of the ACM Internet Measurement Conference (IMC '20). ACM, 712-725.

[66] **[66]** Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. 1986. Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications. ACM Transactions on Programming Languages and Systems 8, 2 (1986), 244-263.

[67] **[67]** Alex Birsan. 2021. Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies. Technical Report. Medium Security Research.

[68] **[68]** Scott A. Crosby and Dan S. Wallach. 2009. Efficient Data Structures for Tamper-Evident Logging. In Proceedings of the 18th USENIX Security Symposium (USENIX Security '09). USENIX Association, 317-334.

[69] **[69]** Rasmus Dahlberg, Tobias Pulls, and Roel Peeters. 2016. Efficient Sparse Merkle Trees: Caching Strategies and Secure (Non-)Membership Proofs. In Proceedings of the 21st Nordic Conference on Secure IT Systems (NordSec '16). Springer, 199-215.

[70] **[70]** Paul Pearce, Roya Ensafi, Frank Li, Nick Feamster, and Vern Paxson. 2017. Global Measurement of DNS Manipulation Using Augur. In Proceedings of the 26th USENIX Security Symposium (USENIX Security '17). USENIX Association, 351-368.

[71] **[71]** Yulong Liu, Frank Li, Baojun Liu, and Haixin Duan. 2018. Measuring Dynamic CDN Geolocation and Anycast Resolution. In Proceedings of the Internet Measurement Conference (IMC '18). ACM, 230-243.

[72] **[72]** Mirian Bouwman, Victor Le Pochat, and Michel van Eeten. 2020. A Longitudinal Study of Threat Intelligence Quality in Commercial Feeds. In Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20). ACM, 1447-1461.

[73] **[73]** Kelly Garrett, Bradley Reaves, and Patrick Traynor. 2019. Detecting Compromised Software Repositories Using Dependency Churn. In Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P '19). IEEE, 1220-1234.

[74] **[74]** Reiner Sailer, Xiaolan Zhang, Trent Jaeger, and Leendert van Doorn. 2004. Design and Implementation of a TCG-based Integrity Measurement Architecture. In Proceedings of the 13th USENIX Security Symposium (USENIX Security '04). USENIX Association, 223-238.

[75] **[75]** Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In Proceedings of the 19th International Conference on World Wide Web (WWW '10). ACM, 661-670.

[76] **[76]** Robin Sommer and Vern Paxson. 2010. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In Proceedings of the 31st IEEE Symposium on Security and Privacy (S&P '10). IEEE, 305-316.

[77] **[77]** David Basin, Cas Cremers, Jannik Dreier, and Ralf Sasse. 2018. Formal Analysis of 5G Authentication. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18). ACM, 1383-1397.

[78] **[78]** Adam Dunkels, Niclas Finne, Joakim Eriksson, and Thiemo Voigt. 2006. Run-Time Dynamic Linking for Sensor Networks. In Proceedings of the 4th International Conference on Embedded Networked Sensor Systems (SenSys '06). ACM, 15-28.

[79] **[79]** Byung-Gon Chun, Petros Maniatis, Scott Shenker, and John Kubiatowicz. 2007. Attested Append-Only Memory: Architecture, Implementation, and Evaluation. In Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07). ACM, 189-204.

[80] **[80]** Jukka Ruohonen and Kalle Hjerppe. 2021. The Problem of Package Signatures in Open-Source Software Repositories. ACM Computing Surveys 54, 4 (2021), 1-36.

[81] **[81]** Christian Rossow, Christian J. Dietrich, Christian Grier, Christian Kreibich, Vern Paxson, Norbert Pohlmann, Herbert Bos, and Maarten van Steen. 2013. Prudent Practices for Designing Malware Experiments. In Proceedings of the 34th IEEE Symposium on Security and Privacy (S&P '13). IEEE, 750-765.

[82] **[82]** Ben Jones, Tzu-Wen Lin, Nick Feamster, and Vern Paxson. 2016. Automated Detection of DNS Censorship Tactics. In Proceedings of the 25th USENIX Security Symposium (USENIX Security '16). USENIX Association, 381-398.

[83] **[83]** Masood Hasan, Sajjad Arshad, and William Robertson. 2020. Triage: Automated Prioritization of Alert Feeds in Enterprise Security Operations. In Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20). ACM, 1205-1219.

[84] **[84]** Yulong Liu, Frank Li, and Vern Paxson. 2015. Characterizing Ephemeral Certificate Revocation in Public Key Infrastructure. In Proceedings of the 2015 ACM Conference on Internet Measurement Conference (IMC '15). ACM, 343-356.

[85] **[85]** Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2023. A Longitudinal Study of Software Supply Chain Attacks in Open Source Ecosystems. ACM Transactions on Privacy and Security 26, 2 (2023), 1-34.

APPENDICES

A.1 Synthetic Observation Record Schema (JSON-LD)

The FLASHWINDOW Layer 4 ledger persists observation events formatted according to a schema-validated JSON-LD standard. Each record links canonical digests, micro-probe signals, invariant evaluation results, and external witness receipts. Listing 1 displays a synthetic observation record captured during an ephemeral tampering event.

LISTING 1: JSON-LD Cryptographic Observation Record Schema

Input: Observation sequence index, target identifier, and cryptographic evidence.
Output: Canonical hash-chained record object.

```
{
"@context": "https://flashwindow.org/schema/v2/record.jsonld",
"record_id": "urn:uuid:7b29a1f4-3d8e-4a6c-9b1e-0a5f8e3c1d2e",
"sequence_number": 1042,
"timestamp_utc": "2026-10-25T14:04:32.184Z",
"target_endpoint": "https://updates.vendor.com/v2/manifest.json",
"vantage_id": "vantage-us-east-as15169",
"tier1_telemetry": {
"http_status": 200,
"etag": ""0x8f2a1b9c"",
"tls_cert_sha256": "0xa1b2c3d4e5f6...",
"bytes_transferred": 324
},
"canonical_manifest": {
"raw_sha256": "0x4f1b8a92...",
"canonical_sha256": "0x9e8d7c6b...",
"download_origin": "https://malicious-mirror.cloud/payload.exe"
},
"invariant_evaluation": {
"violations": ["INV-01: Unauthorized Origin"],
"alert_severity": "CRITICAL"
},
"ledger_chain": {
"prev_record_hash": "0x7c4e2a1b...",
"current_record_hash": "0x2b3c4d5e..."
},
"witness_checkpoint": {
"authority": "FlashWindow Testbed Checkpoint Witness Authority",
"signature_token": "9862c131d34e9206...",
"timestamp_utc": "2026-10-25T14:04:33Z"
}
}
```

A.2 Reference Pseudocode for Cost-Aware Bandit Scheduling

The adaptive prober employs an upper confidence bound (UCB1) multi-armed bandit algorithm to allocate probe rates under daily bandwidth budgets [19]. The scheduler selects action intervals from candidate set A = {60s, 30s, 10s} according to Equation (7). Listing 2 details the scheduling policy selection loop.

LISTING 2: Cost-Aware Bandit Probe Rate Selection Loop

Input: Arms A = {Interval_60s, Interval_30s, Interval_10s}, Historical reward log R, Daily budget B_rem.
Output: Selected sampling interval delta_t.

```
let arms = [Interval_60s, Interval_30s, Interval_10s]
for each arm a in arms do
if pulls[a] == 0 then return a.interval end if
score[a] = mean_reward[a] + c_expl * sqrt(2.0 * ln(total_pulls) / pulls[a]) - gamma_cost * (a.cost / (B_rem + epsilon))
end for
selected_arm = argmax(score)
if selected_arm.estimated_cost > B_rem then
selected_arm = Fallback_Conservative_Arm
end if
return selected_arm.interval
```

A.3 Experimental Protocol Specification and Sensitivity Power Analysis

All experimental protocols, primary metrics, and baseline comparisons follow an explicitly pre-specified evaluation plan documented prior to benchmark execution. For the aggregate 500-campaign evaluation, detecting the observed difference between FLASHWINDOW (0.984) and periodic monitoring (0.888) achieves >99% statistical power (alpha = 0.01). At the individual dwell condition (100 trials at W = 30s), detecting the 48-percentage-point recall advantage achieves >99.9% power (alpha = 0.05). All statistical uncertainty is quantified via standard 95% Wilson score confidence intervals without continuity correction.

A.4 Bounded AI Component Output Validation Schema

All derived outputs generated by the bounded AI assistance plane undergo strict schema validation before presentation to security analysts. The validation parser rejects any summary containing ungrounded factual assertions or attempted modifications to hard invariant flags.